

远程开关量系列通讯协议说明

1. 模块 I/O 寄存器详解

由于 EIO 系列的模块 I/O 控制支持标准的 Modbus，因此模块内部的寄存器将按照 Modbus 通讯协议中的寄存器进行设计的。模块设计了四个寄存器，分别为：类型寄存器、状态寄存器、置高寄存器和置低寄存器。用户可以通过读取“类型寄存器”来了解对应 I/O 的信号类型；读取状态寄存器了解 I/O 信号的输入输出状态；通过写入“置高寄存器”和“置低寄存器”中相应的位来该表其对应 I/O 的输出状态。如下表：

序号	地址	名称	类型	命令	描述
1	0x0000	type	RO	0x03 和 0x04	I/O 信号类型寄存器（1 为输出，0 为输入）
2	0x0001	state	RO	0x03 和 0x04	I/O 口当前状态寄存器（1 为高电平，0 为低电平）
3	0x0002	setH	WR	0x06 和 0x10	I/O 口置高寄存器（对应通道设为 1 触发为断开状态）
4	0x0003	setL	WR	0x06 和 0x10	I/O 口置低寄存器（对应通道设为 1 触发为导通状态）

- a. 数字前面的“0x”代表这个数值的类型为 16 进制
- b. RO 代表这个寄存器为只读寄存器，用户如果向这个寄存器写数据将出现报错
- c. WR 代表这个寄存器为只写寄存器，用户读取这个寄存器，读取的数值不代表任何意义

1.1. 信号类型寄存器

信号类型寄存器里面包含有 I/O 信号的信号类型，高电平（1）代表信号类型为输出，低电平（0）代表信号类型为输入。

Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Ch16	Ch15	Ch14	Ch13	Ch12	Ch11	Ch10	Ch9	Ch8	Ch7	Ch6	Ch5	Ch4	Ch3	Ch2	Ch1

Bit0 代表 I/O 信号通道 1，Bit15 代表 I/O 信号通道 16，这个系列中有些模块的 I/O 通道数小于 16，譬如 EI01010 为 10 通道，则读这个寄存器中的 Bit10 到 Bit15 的值为不确定状态是无效的。

1.2. 状态寄存器

状态寄存器的信息为 I/O 信号的状态信息，“1”代表为高电平、“0”代表为低电平。当 I/O

信号为输入时这个信息反映对应 I/O 的当前输入状态，为输出时则反映为 I/O 的输出电平。寄存器信息如下表：

Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Ch16	Ch15	Ch14	Ch13	Ch12	Ch11	Ch10	Ch9	Ch8	Ch7	Ch6	Ch5	Ch4	Ch3	Ch2	Ch1

Bit0 代表 I/O 信号通道 1，Bit15 代表 I/O 信号通道 16，这个系列中有些模块的 I/O 通道数小于 16，譬如 EI01010 为 10 通道，则读这个寄存器中的 Bit10 到 Bit15 的值为不确定状态是无效的。

1.3. 置高寄存器

置高寄存器用来设置对应 I/O 信号的端口为高电平（1），当用户写这个寄存器时，寄存器中为“1”的位对应的 I/O 将设置为高电平（1），为“0”的位对应的 I/O 的输出状态将不改变，另外当这个位对应的 I/O 类型为输入时，写入的数值将不会影响其输入电平。如下表：

Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Ch16	Ch15	Ch14	Ch13	Ch12	Ch11	Ch10	Ch9	Ch8	Ch7	Ch6	Ch5	Ch4	Ch3	Ch2	Ch1

Bit0 代表 I/O 信号通道 1，Bit15 代表 I/O 信号通道 16，这个系列中有些模块的 I/O 通道数小于 16，譬如 EI01010 为 10 通道，则写这个寄存器中的 Bit10 到 Bit15 的值是无意义的，模块不对这些值做任何操作。

1.4. 置低寄存器

置低寄存器用来设置对应 I/O 信号的端口为低电平（0），当用户写这个寄存器时，寄存器中为“1”的位对应的 I/O 将设置为低电平（0），为“0”的位对应的 I/O 的输出状态将不改变，另外当这个位对应的 I/O 类型为输入时，写入的数值将不会影响其输入电平。如下表：

Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Ch16	Ch15	Ch14	Ch13	Ch12	Ch11	Ch10	Ch9	Ch8	Ch7	Ch6	Ch5	Ch4	Ch3	Ch2	Ch1

Bit0 代表 I/O 信号通道 1，Bit15 代表 I/O 信号通道 16，这个系列中有些模块的 I/O 通道数小于 16，譬如 EI01010 为 10 通道，则写这个寄存器中的 Bit10 到 Bit15 的值是无意义的，

模块不对这些值做任何操作。

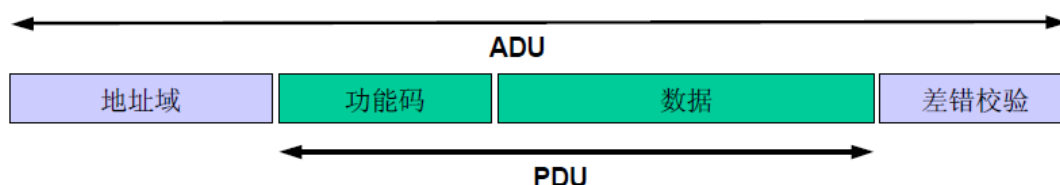
2. 模块 IO 通讯协议详解

此系列模块同时支持以太网和 RS485 两种通讯方式，由于以太网和 RS485 在 modbus 协议中的命令包组织方式不同，所以这个系列的模块通讯协议有两种：stand 和 modbus。其中“stand”指的是 modbus RTU 适用于 RS485 和以太网，modbus 指的是 modbus TCP 适用于以太网且以太网的工作模式为 TcpServer。

2.1. stand（标准）协议

这个模块中的“stand/标准”协议指的是 Modbus RTU。当用户使用 RS485 或以以太网的工作模式为“TcpClient”、“UDP”和“TcpServer”且配置协议为标准时，模块采用的通讯协议就是这个。通讯首先由主机发起，模块进行响应。下面将对 Modbus RTU 这个协议进行详细介绍。

Modbus RTU 命令包格式如下：



- **地址域：**为设备的地址，这个参数在 RS485 通讯方式中必须和配置参数中的设备地址相同；在以太网通讯格式中这个参数没有什么意义，模块在通讯过程中不使用这个参数，用户可随意指定。
- **功能码：**此系列模块支持的功能码有：0x03、0x04、0x06 和 0x10；
- **数据：**为对应功能码的参数数据
- **差错校验：**此协议试用的校验为 CRC 校验，CRC 校验生成的校验值为一个 16bit 的双字节数据，在命令包中的排布为低字节在前，高字节在后。如 0x1234 则发送的第一个校验字节为 0x34 第二个字节为 0x12。校验函数参照附录。

注：modbus 使用“大端”的数据表示方式来表示数据和地址项，这意味着发送多个字节时首先发送最高有效位。如：0x1234 发送的第一个字节为 0x12 然后为 0x34。

2.1.1 功能码 0x03 读取（类型或状态）寄存器

modbus 功能码 0x03 是用来读取保持寄存器连续块的内容，本模块系列通过此命令可以读取所有寄存器，但读取“置高寄存器”和“置低寄存器”是没有意义的，一般用这个命令读取寄存器“类型寄存器”和“”状态寄存器。其通讯格式如下：

主机请求

功能码	一个字节	0x03
起始地址	两个字节	0x0000~0x0003
寄存器数量	两个字节	0x0001~0x0004 (N)

模块响应

功能码	一个字节	0x03
字节数	一个字节	N * 2
寄存器内容	N * 2	

如 用
户 使

用标准协议读取模块类型寄存器和状态寄存器，如下：

请求		响应		备注
域名	值	域名	值	
地址	0x01	地址	0x01	标准协议用
功能码	0x03	功能码	0x03	标准和 modbus 均用
起始地址高	0x00	返回字节数	0x04	标准和 modbus 均用
起始地址低	0x00	类型寄存器高	0x03	标准和 modbus 均用
寄存器数量高	0x00	类型寄存器低	0xfe	标准和 modbus 均用
寄存器数量低	0x02	状态寄存器高	0x00	标准和 modbus 均用
		状态寄存器低	0x01	标准和 modbus 均用
CRC 校验低	0xc4	CRC 校验低	0x5a	标准协议用
CRC 校验高	0x0b	CRC 校验高	0x47	标准协议用

在这个例子中读取的类型寄存器的值为 0x03fe，状态寄存器的值为 0x0001。

2.1.2. 功能码 0x04 读取（类型或状态）寄存器

modbus 功能码 0x04 是用来读取连续的输入寄存器，在本模块中可以通过此命令读取“类型寄存器”和“状态寄存器”，其通讯格式如下：

主机请求

功能码	一个字节	0x04
起始地址	两个字节	0x0000 ~ 0x0001

寄存器数量	两个字节	0x0001 ~ 0x0002 (N)
-------	------	---------------------

模块响应

功能码	一个字节	0x04
字节数	一个字节	N * 2
寄存器内容	N * 2	

如用户使用这个功能码读取模块的“类型寄存器”和“状态寄存器”，如下：

请求		响应		备注
域名	值	域名	值	
地址	0x01	地址	0x01	标准协议用
功能码	0x04	功能码	0x04	标准和 modbus 均用
起始地址高	0x00	返回字节数	0x04	标准和 modbus 均用
起始地址低	0x00	类型寄存器高	0x03	标准和 modbus 均用
寄存器数量高	0x00	类型寄存器低	0xfe	标准和 modbus 均用
寄存器数量低	0x02	状态寄存器高	0x00	标准和 modbus 均用
		状态寄存器低	0x01	标准和 modbus 均用
CRC 校验低	0x71	CRC 校验低	0x5b	标准协议用
CRC 校验高	0xcb	CRC 校验高	0xf0	标准协议用

在这个例子中读回的类型寄存器的值为 0x03fe，状态寄存器的值为 0x0001。

2.1.3. 功能码 0x06 写单个（类型或状态）寄存器

Modbus 中此功能码是用来写单个保持寄存器的，此系列模块可通过此命令更改“置高寄存器”或“置低寄存器”的值。其通讯数据格式如下：

主机请求

功能码	一个字节	0x06
-----	------	------

寄存器地址	两个字节	0x0002 或 0x0003
寄存器值	两个字节	

模块响应

功能码	一个字节	0x06
寄存器地址	两个字节	0x0002 或 0x0003
寄存器值	N * 2	

例如，通过此命令更改模块“置高寄存器”的内容，如下：

请求		响应		备注
域名	值	域名	值	
地址	0x01	地址	0x01	标准协议用
功能码	0x06	功能码	0x06	标准和 modbus 均用
寄存器地址高	0x00	寄存器地址高	0x00	标准和 modbus 均用
寄存器地址低	0x02	寄存器地址低	0x02	标准和 modbus 均用
寄存器值高	0x03	寄存器值高	0x03	标准和 modbus 均用
寄存器值低	0xfe	寄存器值低	0xfe	标准和 modbus 均用
CRC 校验低	0xa9	CRC 校验低	0xa9	标准协议用
CRC 校验高	0x7a	CRC 校验高	0x7a	标准协议用

在这个命令中将“置高寄存器”的值设为 0x03fe，则模块 I/O 通道 2 到 10 将置为高电平。

2.1.4. 功能码 0x10 写连续的多个保持寄存器

Modbus 中此功能码是用来写连续的多个保持寄存器，此系列模块可以通过这个功能码同时写“置高寄存器”和“置低寄存器”来完成 I/O 状态的设定。其通讯数据格式如下：

主机请求

功能码	一个字节	0x10
起始地址	两个字节	0x0002~0x0003

寄存器数量	两个字节	1 ~2 (N)
字节数	一个字节	2 或 4
寄存器值	N * 2 个字节	

模块响应

功能码	一个字节	0x10
起始地址	两个字节	0x0002~0x0003
寄存器数量	两个字节	1~2 (N)

例

如用这个命令同时写“置高寄存器”和“置低寄存器”，将模块 I/O 通道 2 和 3 置高、通道 4 和 5 置低。其通讯内容如下：

请求		响应		备注
域名	值	域名	值	
地址	0x01	地址	0x01	标准协议用
功能码	0x10	功能码	0x10	标准和 modbus 均用
起始地址高	0x00	起始地址高	0x00	标准和 modbus 均用
起始地址低	0x02	起始地址低	0x02	标准和 modbus 均用
寄存器数量高	0x00	寄存器数量高	0x00	标准和 modbus 均用
寄存器数量低	0x02	寄存器数量低	0x02	标准和 modbus 均用
字节数	0x04			标准和 modbus 均用
置高寄存器高	0x00			标准和 modbus 均用
置高寄存器低	0x06			标准和 modbus 均用
置低寄存器高	0x00			标准和 modbus 均用
置低寄存器低	0x18			标准和 modbus 均用
CRC 校验低	0x92	CRC 校验低	0xe0	标准协议用
CRC 校验高	0x7d	CRC 校验高	0x08	标准协议用

2.1.5. 功能码 0x01 读取线圈状态

Modbus 协议中用此命令来读取线圈 1 至 2000 的连续状态，用户可以通过此命令来读取模块上 I/O 通道的当前状态，I/O 通道的序号编码从 1 开始，对应的地址编码从 0 开始。

主机请求

功能码	一个字节	0x01
起始地址	两个字节	0x0000~0x000F
线圈数量	两个字节	最大值为 16，和模块支持的通道数一致

模块响应

功能码	一个字节	0x01
字节数	一个字节	N（N=输出数量/8，如果余数不等于 0，那么 N=N+1）
线圈状态	N 个字节	

如果用这个命令读取 I/O 通道 1 到 5 的状态。其通讯内容如下：

请求		响应		备注
域名	值	域名	值	
地址	0x01	地址	0x01	标准协议用
功能码	0x01	功能码	0x01	标准和 modbus 均用
起始地址高	0x00	字节数	0x01	标准和 modbus 均用
起始地址低	0x00	线圈状态	0x1f	标准和 modbus 均用
线圈数量高	0x00			标准和 modbus 均用
线圈数量低	0x05			标准和 modbus 均用
CRC 校验低	0xfc	CRC 校验低	0x40	标准协议用
CRC 校验高	0x09	CRC 校验高	0x10	标准协议用

2.1.6. 功能码 0x02 读取离散量输入连续状态

Modbus 使用该功能码读取离散量输入的 1 至 2000 连续状态。用户也可以通过这个命令读取模块上对应 I/O 通道的状态，I/O 通道的序号编码从 1 开始，对应的地址编码从 0 开始。

主机请求

功能码	一个字节	0x02
起始地址	两个字节	0x0000~0x000F
线圈数量	两个字节	最大值为 16，和模块支持的通道数一致

模块响应

功能码	一个字节	0x02
字节数	一个字节	N（N=输出数量/8，如果余数不等于 0，那么 N = N+1）
线圈状态	N 个字节	

例

如用这个命令读取 I/O 通道 1 到 5 的状态。其通讯内容如下：

请求		响应		备注
域名	值	域名	值	
地址	0x01	地址	0x01	标准协议用
功能码	0x02	功能码	0x02	标准和 modbus 均用
起始地址高	0x00	字节数	0x01	标准和 modbus 均用
起始地址低	0x00	线圈状态	0x1f	标准和 modbus 均用
线圈数量高	0x00			标准和 modbus 均用
线圈数量低	0x05			标准和 modbus 均用
CRC 校验低	0xB8	CRC 校验低	0x40	标准协议用
CRC 校验高	0x09	CRC 校验高	0xE0	标准协议用

2.1.7. 功能码 0x05 写单个线圈输出为 ON 或 OFF

Modbus 使用该功能码写单个线圈输出为 ON 或 OFF。用户可以通过这个命令来设置模块上对应通道的输出状态。I/O 通道的序号编码从 1 开始，对应的地址编码从 0 开始。

主机请求

功能码	一个字节	0x05
-----	------	------

线圈地址	两个字节	0x0000~0x000F
线圈状态	两个字节	0x0000（置对应 I/O 为低电平）或 0xff00（置对应 I/O 为高电平）

模块响应

功能码	一个字节	0x05
线圈地址	两个字节	0x0000~0x000F
线圈状态	两个字节	0x0000 或 0xff00

例如用这个命令来设置 I/O 通道 5 为低电平。其通讯内容如下：

请求		响应		备注
域名	值	域名	值	
地址	0x01	地址	0x01	标准协议用
功能码	0x05	功能码	0x05	标准和 modbus 均用
线圈地址高	0x00	线圈地址高	0x00	标准和 modbus 均用
线圈地址低	0x04	线圈地址低	0x04	标准和 modbus 均用
线圈状态高	0x00	线圈状态高	0x00	标准和 modbus 均用
线圈状态低	0x00	线圈状态低	0x00	标准和 modbus 均用
CRC 校验低	0x8C	CRC 校验低	0x0B	标准协议用
CRC 校验高	0x0B	CRC 校验高	0x8C	标准协议用

2.1.8 功能码 0x0f 强制线圈序列中的每个线圈为 ON 或 OFF

Modbus 使用该功能码强制线圈序列中的每个线圈为 ON 或 OFF。用户可以通过这个命令来设置模块上相连多个 I/O 通道的输出状态。I/O 通道的序号编码从 1 开始，对应的地址编码从 0 开始。

主机请求

功能码	一个字节	0x0F
-----	------	------

线圈地址	两个字节	0x0000~0x000F
线圈数量	两个字节	最大值为 16，这个要和模块支持的通道数一致
字节数	一个字节	N(N=输出数量/8，如果余数不等于 0，那么 N=N+1)
线圈状态	N 个字节	

模块响应

功能码	一个字节	0x0f
线圈地址	两个字节	和主机请求中的对应项一致
线圈数量	两个字节	和主机请求中的对应项一致

例如用这个命令来设置 I/O 通道 1~5 为高电平。其通讯内容如下：

请求		响应		备注
域名	值	域名	值	
地址	0x01	地址	0x01	标准协议用
功能码	0x0f	功能码	0x0f	标准和 modbus 均用
线圈地址高	0x00	线圈地址高	0x00	标准和 modbus 均用
线圈地址低	0x00	线圈地址低	0x00	标准和 modbus 均用
线圈数量高	0x00	线圈数量高	0x00	标准和 modbus 均用
线圈数量低	0x05	线圈数量低	0x05	标准和 modbus 均用
字节数	0x01			
线圈状态	0x1f			
CRC 校验低	0x2e	CRC 校验低	0xc8	标准协议用
CRC 校验高	0x9e	CRC 校验高	0x95	标准协议用

2.1.9. 功能码通讯出错

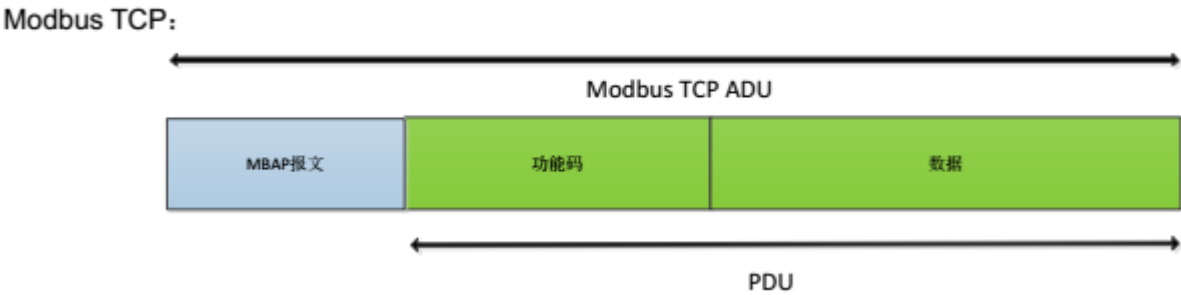
当通讯出现硬件连接失败或命令包模块不支持则会出现通讯失败。如果硬件连接失败或出现通讯干扰造成数据传输失败，这个时候用户发出的命令模块不会响应，这时用户需要通过超时判断来识别通讯失败；如果用户组织的命令包模块不支持则模块将向用户返回其错误代码，

出现错误时模块返回的功能码为原功能码或上 0x80（譬如功能码 0x03 出错时返回的功能码为 0x83），其后跟上错误类型，其错误类型代码如下表：

序号	类型代码	含义
1	0x01	用户发送的功能码，模块不支持
2	0x02	用户组织的地址出现错误
3	0x03	用户组织的数据出现错误

2. 2. Modbus TCP 协议

该模式的协议只能通过以太网且以太网工作模式为“TcpServer”时才能使用。其通讯数据包如下：



通讯时首先用户的设备需要通过以太网和模块建立连接，用户的设备工作在 TCP 客户端，模块工作在服务器。通讯首先有用户的设备发起，模块进行响应。

MBAP 报文头格式如下：

域	长度	描述
事务元标示符	两个字节	Modbus 请求响应的识别码
协议标示符	两个字节	0=Modbus 协议
长度	两个字节	以下字节的数量
单元标示符	一个字节	串行链路或其他总站连接的从站识别码

- **事务元标示符：**用于事务处理配对，模块返回时直接将这个域的内容复制返回
- **协议标示符：**这个域的值设置为 0 来代表 modbus 协议
- **长度：**指下一个域的字节数，包含有“单元标示符”、“功能码”和“数据”
- **单元标示符：**这个域的内容在模块通讯中不起作用用户可随意指定

这个协议中的“功能码”和“数据”和标准协议中的对应项相同，具体请参考前面章节。